General Review

Prepared by Yaren DERİCİ,Taylan AKBAŞ,Ege Erberk USLU

SE 2228 - ANALYSIS AND DESIGN OF ALGORITHMS 2021-2022 Fall

Table of Contents

- Pointers
  - Pointer Syntax and Examples
  - Double Pointer
- Dynamic Memory Allocation
  - Calloc
  - Malloc
  - Free
  - Realloc

Pointers

Pointers (pointer variables) are special variables that are used to store addresses rather than values.

Pointer Syntax

Here, we have declared a pointer p of int type.

```
int* p;
int *p1;
int * p2;
```

Types of Pointers

- char*
- int*
- float*
- double*
- void*

- Size of all pointers:
  - In 32 bit processors, 4 byte.
  - In 64-bit processors, 8 byte.

Let’s take an example

- Assigning addresses to Pointers

```
int c = 10

int *p;

p = &c;
```

Here, 10 is assigned to the *c* variable. And, the address of *c* is assigned to the *p* pointer.

Get Value of Thing Pointed by Pointers

```
int c = 10

int *p;

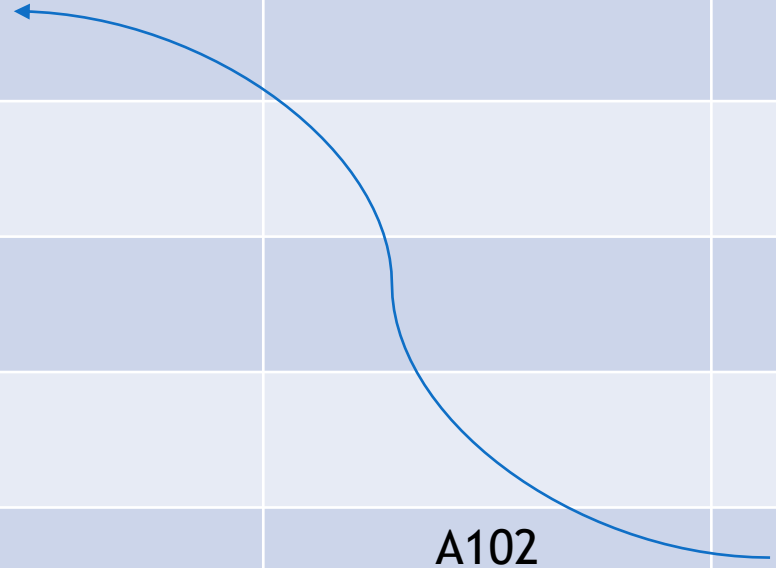
p = &c;

printf("%d", *p);      // Output: 10
```

To get the value of the thing pointed by the pointers, we use the * operator.

To get the value stored in that address, we used *p.

Address Value	Value	Variable Name
A101		
A102	10	a
A103		
A104		
A105		
A106		
A107	A102	P



```
int a = 10;
int *p ;
p=&a;

printf(‘’%d”,*p); //10
printf(‘’%d”,p);  //A102
printf(‘’%d”,&p); //A107
```

Changing Value Pointed by Pointers

- Let's take an example.

```
int c = 10

İnt *p;

p = &c;

c = 1;

printf("%d", c); // Output: 1

printf("%d", *p); // Output: 1
```

We have assigned the address of *c* to the *p* pointer.

Then, we changed the value of *c* to 1. Since *p* and the address of *c* is the same, **p* gives us 1.

Let's take another example.

```
int *p, c;

c = 5;

p = &c;

*p = 1;

printf("%d", *p); // Ouptut: 1

printf("%d", c); // Output: 1
```

We have assigned the address of *c* to the *p* pointer.

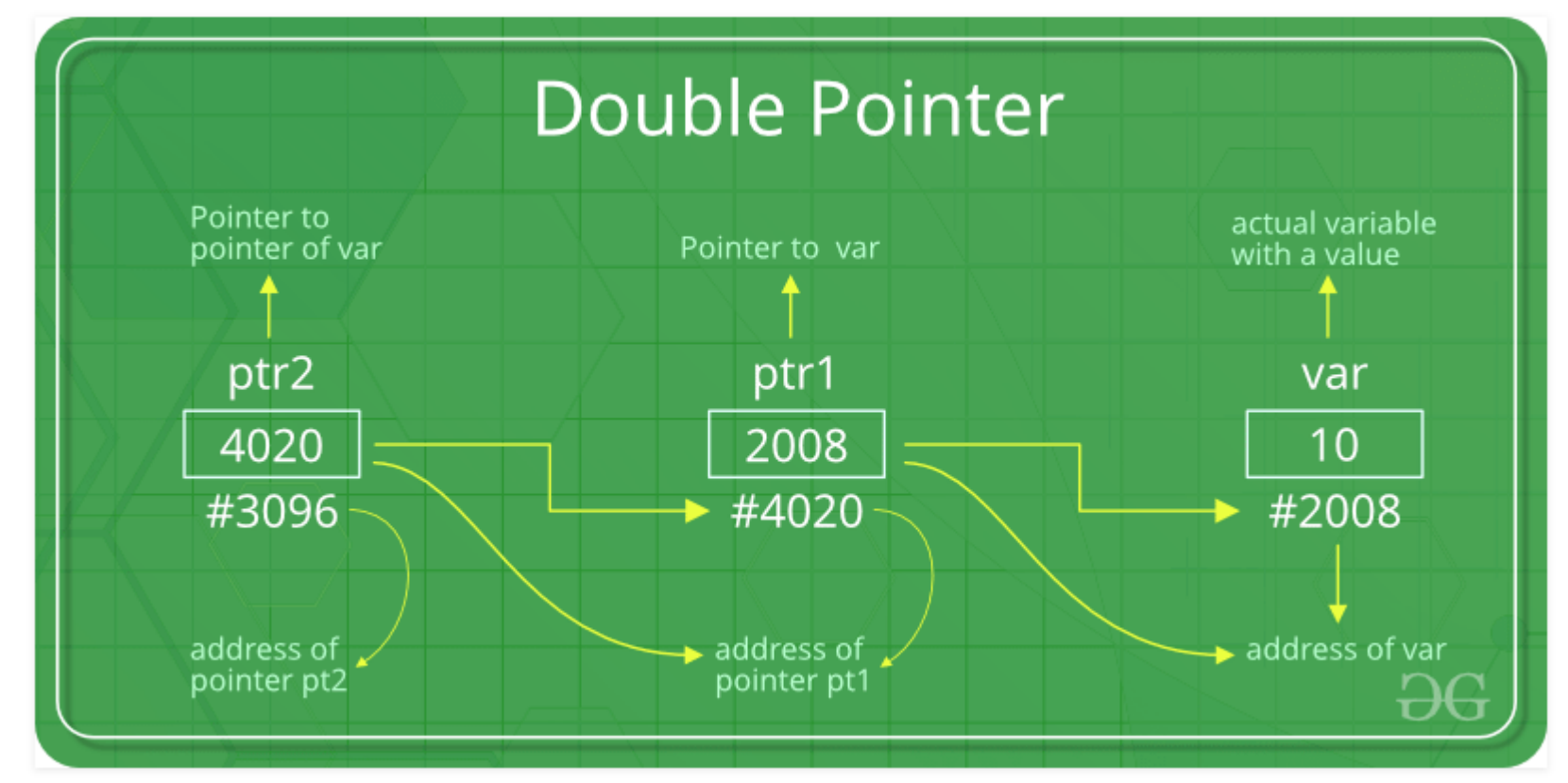
Then, we changed *p to 1 using *p = 1; .

Since *p* and the address of *c* is the same, *c* will be equal to 1.

Double Pointer (Pointer to Pointer)

- How to declare a pointer to pointer in C?

```
int **ptr;
```



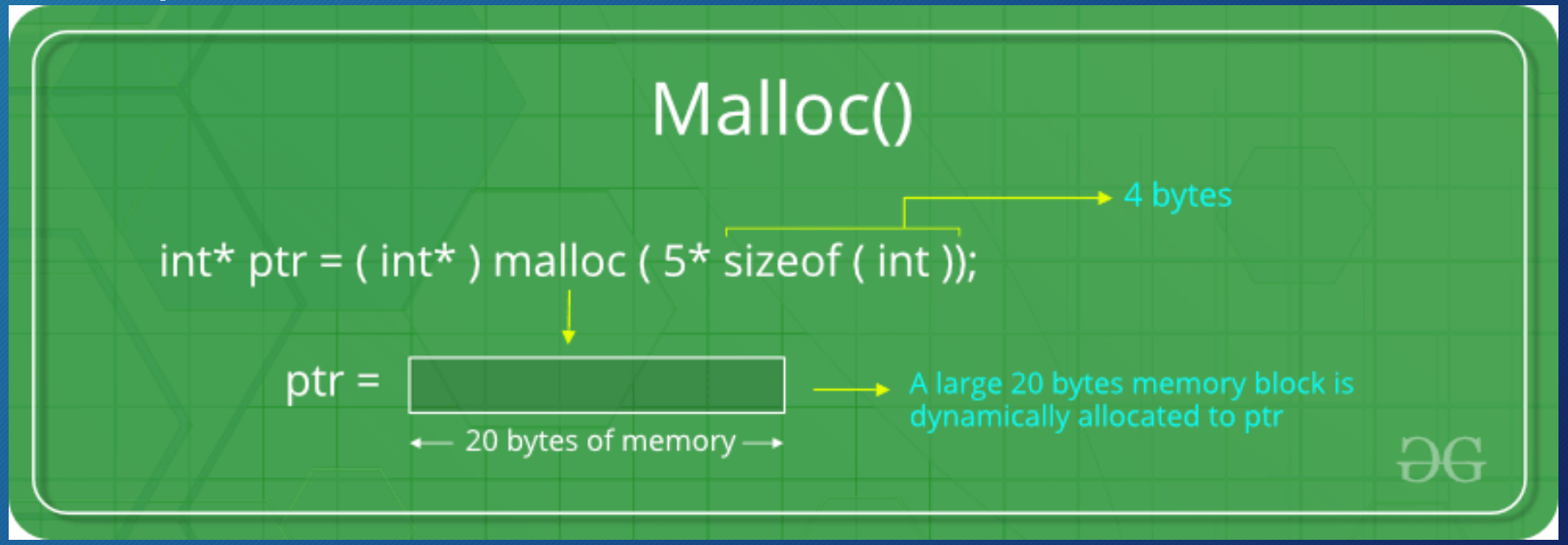
Dynamic Memory Allocation

There are 4 library functions provided by C defined under <stdlib.h>

- *1. malloc()*
- *2. calloc()*
- *3. realloc()*
- *4. free()*

Malloc()

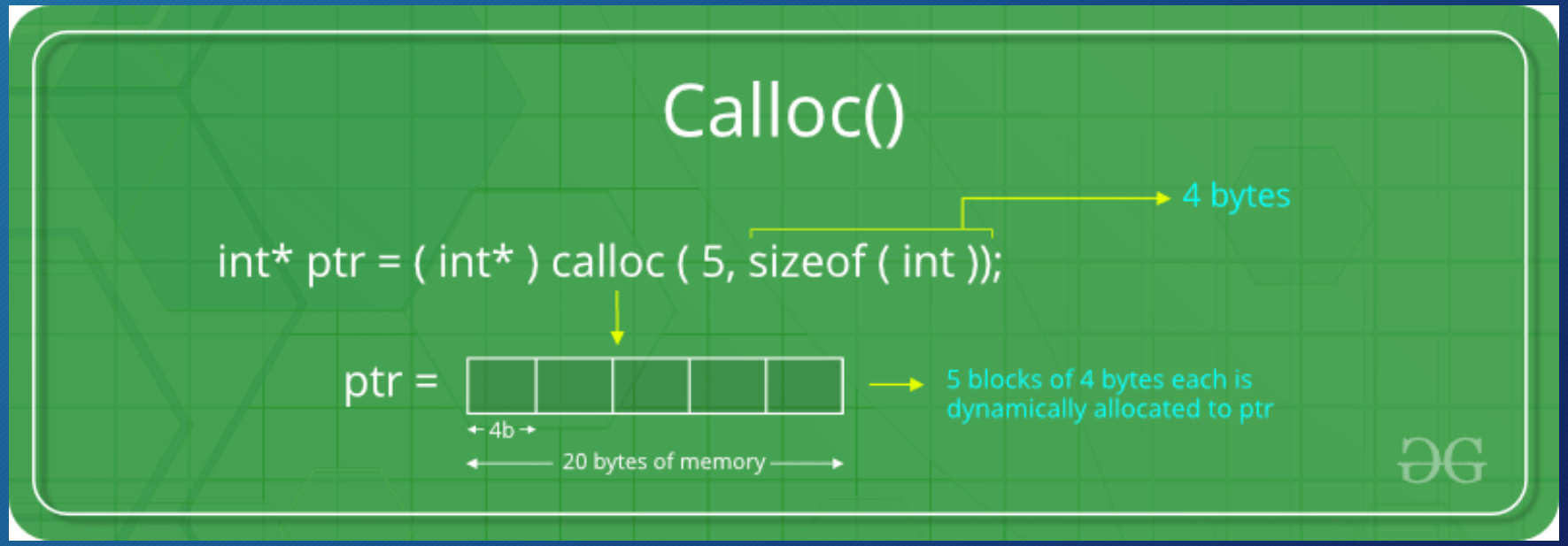
- “**malloc**” or “**memory allocation**” method in C is used to dynamically allocate a single large block of memory with the specified size.



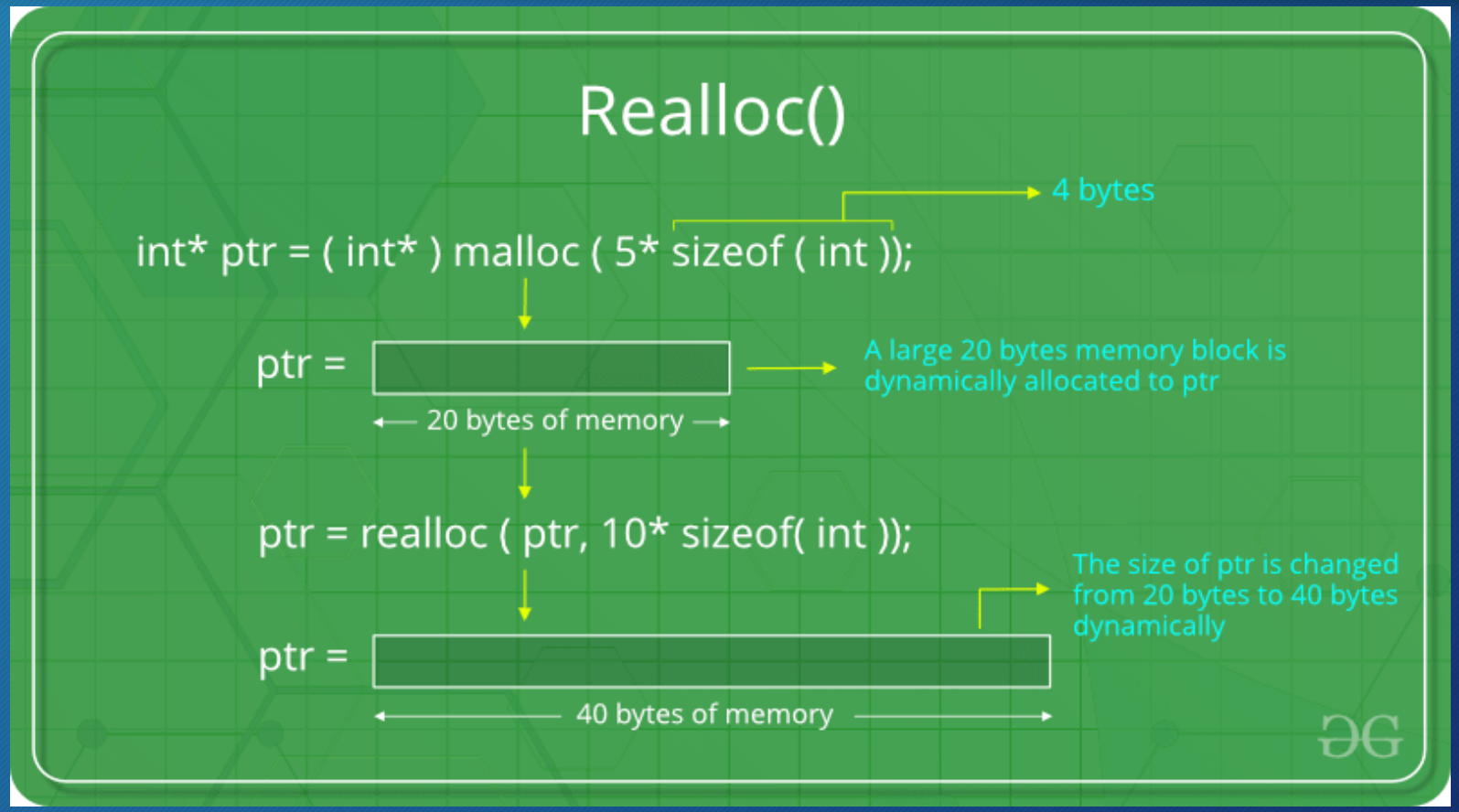
Note: Since malloc is a void function,so we have to use type casting.(int *)

Calloc()

- “**calloc**” or “**contiguous allocation**” method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. It initializes each block with a default value ‘0’.

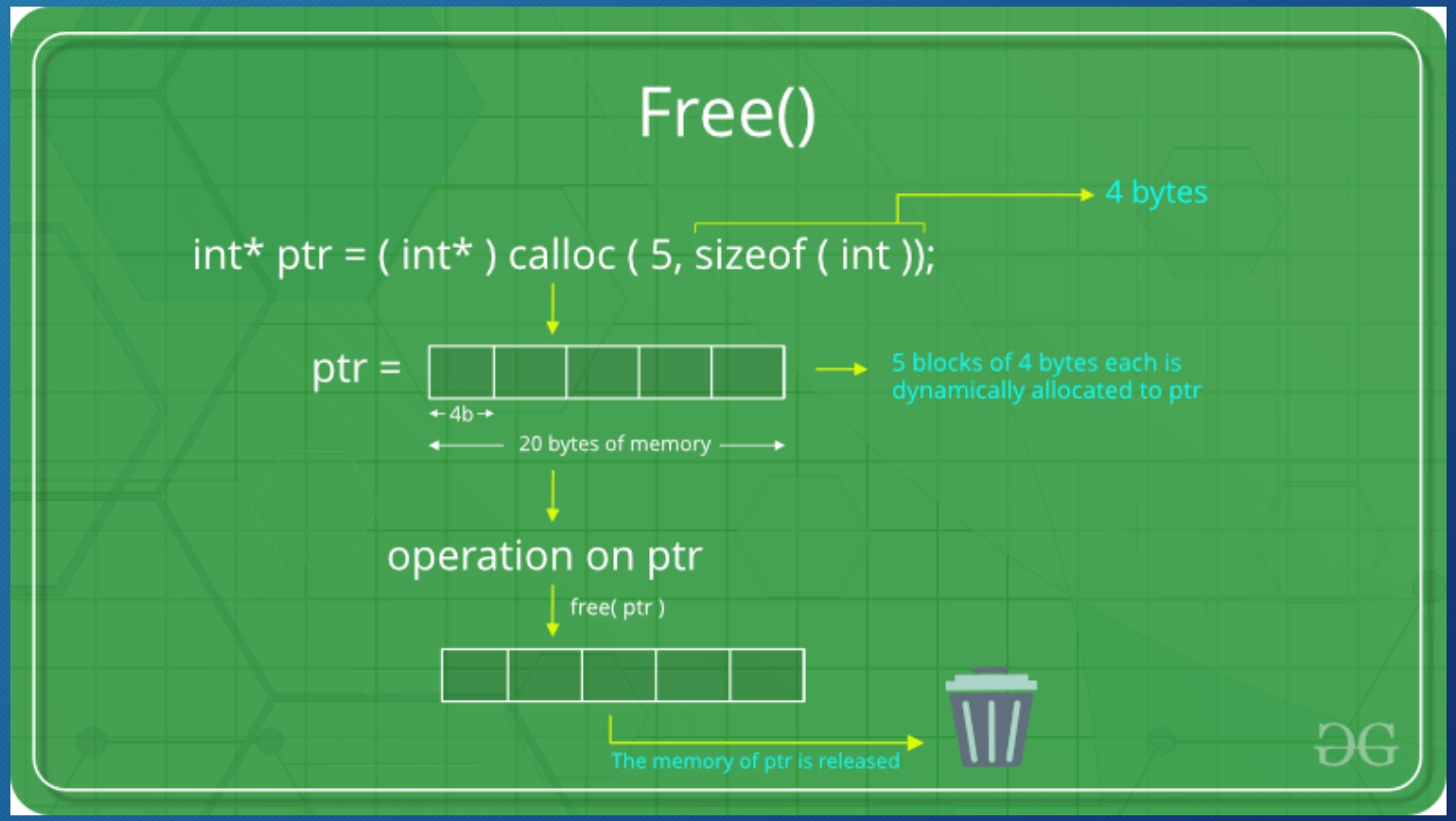


Realloc()



“realloc” or **“re-allocation”** method in C is used to dynamically change the memory allocation of a previously allocated memory. In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory.**

Free()



"**free**" method in C is used to dynamically **de-allocate** the memory. The memory allocated using functions malloc() and calloc() is not de-allocated on their own.It helps to reduce wastage of memory by freeing it.

References

- https://www.geeksforgeeks.org/double-pointer-pointer-pointer-c/
- https://www.geeksforgeeks.org/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/